Table of Contents

..... 1
---- SIMULATION PARAMETERS ---------- 2
---- MISSILE PHYSICAL PARAMETERS ---------- 2
---- AERODYNAMIC COEFFICIENT TABLE (simplified) ---------- 2
---- INITIAL CONDITIONS ---------- 2
---- TARGET INITIAL CONDITIONS ---------- 3
---- ADRC CONTROLLER PARAMETERS (pitch axis) ---------- 3
---- GUIDANCE LAW PARAMETERS ---------- 3
---- PRE-ALLOCATE STATE HISTORY ---------- 3
---- INITIALIZE STATE ---------- 4
======== MAIN SIMULATION LOOP ========== 4
---- PLOT RESULTS ---------- 6
========== 11


```
% ==========================================================================
% MAIN_SIMULATION.M
% 6-DOF Missile Guidance and Control System
% Active Disturbance Rejection Control (ADRC) + Proportional Navigation
%
% Coordinate Frames:
%   Body Frame (B): Origin at missile CG, x-axis forward (nose), y-axis
%                   right, z-axis down (right-hand system)
%   Inertial Frame (I): NED (North-East-Down) Earth-fixed frame
%
% State Vector (12 states):
%   [u, v, w]      - Body-frame linear velocities (m/s)
%   [p, q, r]      - Body-frame angular rates (rad/s)
%   [phi, theta, psi] - Euler angles: roll, pitch, yaw (rad)
%   [x, y, z]      - Inertial position: North, East, Down (m)
%
% Author: Aerospace Control Systems Engineer
% ==========================================================================

clear; clc; close all;

fprintf('==============================================================\n');
fprintf('   6-DOF Missile Guidance & Control Simulation (ADRC + PN)  \n');
fprintf('==============================================================\n\n');

==============================================================
   6-DOF Missile Guidance & Control Simulation (ADRC + PN)
==============================================================
```

---- SIMULATION PARAMETERS ----------

```
dt      = 0.005;          % Time step (s) — 200 Hz for numerical stability
t_end   = 25.0;           % Total simulation time (s)
t       = 0:dt:t_end;     % Time vector
N_steps = length(t);
```

---- MISSILE PHYSICAL PARAMETERS ----------

```
missile.mass    = 120;        % kg
missile.Ixx     = 0.5;        % kg·m²  (roll  moment of inertia)
missile.Iyy     = 80;         % kg·m²  (pitch moment of inertia)
missile.Izz     = 80;         % kg·m²  (yaw   moment of inertia)
missile.Ixz     = 0.0;        % kg·m²  (cross product term — symmetric)
missile.Sref    = 0.02;       % m²     (reference area = π·d²/4, d≈0.16m)
missile.Lref    = 0.16;       % m      (reference length = body diameter)
missile.xcg     = 1.2;        % m      (CG from nose)
missile.thrust  = 5000;       % N      (constant thrust during burn)
missile.t_burn  = 8.0;        % s      (motor burn time)
```

---- AERODYNAMIC COEFFICIENT TABLE (simplified) ----------

These are Mach/alpha-independent simplified coefficients for clarity. In a real system these come from a wind-tunnel lookup table.

```
aero.CD0   =  0.30;      % Zero-lift drag coefficient
aero.CDa   =  1.80;      % Drag due to alpha (per rad²)
aero.CLa   =  12.0;      % Lift curve slope (per rad)
aero.CYb   = -10.0;      % Side-force curve slope (per rad)
aero.Cmq   = -20.0;      % Pitch damping derivative (per rad/s)
aero.Cmde  =  2.5;       % Pitch moment due to elevator deflection (per rad)
aero.Cma   = -2.0;       % Static pitch stability (per rad)  (<0 = stable)
aero.Clp   = -0.5;       % Roll damping derivative
aero.Cnr   = -0.5;       % Yaw  damping derivative
missile.aero = aero;
```

---- INITIAL CONDITIONS ----------

Body velocities

```
u0   = 300;   v0 = 0;   w0 = 0;       % m/s (initial speed along x-body)
% Angular rates
p0   = 0;     q0 = 0;   r0 = 0;       % rad/s
```

```
% Euler angles
phi0   = 0;   theta0 = deg2rad(5);   psi0 = 0;  % slight nose-up pitch
% Inertial position (NED), missile launches from origin
x0 = 0;  y0 = 0;  z0 = -500;         % z negative = 500 m altitude (NED)

X0 = [u0; v0; w0; p0; q0; r0; phi0; theta0; psi0; x0; y0; z0];
```

---- TARGET INITIAL CONDITIONS ----------

```
target.pos    = [8000; 500; -500];    % NED (m): 8 km ahead, 500 m East,
same alt
target.vel    = [-150; 0; 0];         % m/s (approaching head-on)
```

---- ADRC CONTROLLER PARAMETERS (pitch axis) ----------

Third-order ESO bandwidth: omega_o controls observer speed. Controller bandwidth: omega_c controls closed-loop speed. Rule of thumb: omega_o = (3~10) * omega_c

```
adrc.b0      = 1/missile.Iyy;   % Control gain estimate (1/I_yy)
adrc.omega_c = 8.0;              % Controller bandwidth (rad/s)
adrc.omega_o = 50.0;             % Observer  bandwidth (rad/s)

% PD gains from bandwidth parameterization
adrc.k1 = adrc.omega_c^2;        % Proportional gain
adrc.k2 = 2*adrc.omega_c;        % Derivative gain

% ESO gains (place observer poles at -omega_o)
adrc.beta1 = 3*adrc.omega_o;
adrc.beta2 = 3*adrc.omega_o^2;
adrc.beta3 = adrc.omega_o^3;

% Elevator deflection limits
adrc.de_max = deg2rad(25);        % Maximum deflection ±25°
```

---- GUIDANCE LAW PARAMETERS ----------

```
guidance.N  = 4.0;                % Navigation constant (3-5 typical)
```

---- PRE-ALLOCATE STATE HISTORY ----------

```
X_hist      = zeros(12, N_steps);
de_hist     = zeros(1,  N_steps);   % Elevator deflection (rad)
theta_cmd   = zeros(1,  N_steps);   % Pitch command from guidance (rad)
```

```
dist_est    = zeros(1,  N_steps);   % ESO disturbance estimate
target_hist = zeros(3,  N_steps);   % Target position history
miss_dist   = zeros(1,  N_steps);   % Miss distance history
```

---- INITIALIZE STATE ----------

```
X           = X0;
X_hist(:,1) = X0;

% ESO internal states: z1=theta_est, z2=q_est, z3=disturbance_est
eso_state   = [X0(8); X0(5); 0];   % [z1, z2, z3]

% Guidance memory (LOS rate filter)
lambda_prev = 0;
t_prev      = 0;
lambda_dot  = 0;

fprintf('Initial missile position: [%.0f, %.0f, %.0f] m (NED)\n', X0(10),
X0(11), X0(12));
fprintf('Initial missile speed: %.1f m/s\n', norm(X0(1:3)));
fprintf('Target initial position: [%.0f, %.0f, %.0f] m (NED)\n\n',
target.pos);
fprintf('Running simulation...\n');
```

```
Initial missile position: [0, 0, -500] m (NED)
Initial missile speed: 300.0 m/s
Target initial position: [8000, 500, -500] m (NED)

Running simulation...
```

======== MAIN SIMULATION LOOP ========== ====

```
for k = 1:N_steps-1
    tk = t(k);

    % ---- Current state extraction ----
    u=X(1); v=X(2); w=X(3);
    p=X(4); q=X(5); r=X(6);
    phi=X(7); theta=X(8); psi=X(9);
    xI=X(10); yI=X(11); zI=X(12);

    % ---- Airspeed and aerodynamic angles ----
    V     = sqrt(u^2 + v^2 + w^2);
    V     = max(V, 1.0);              % Avoid division by zero
    alpha = atan2(w, u);              % Angle of attack (rad)
    beta  = asin(v / V);              % Sideslip angle (rad)
```

```
% ---- Dynamic pressure ----
rho  = atmosphere_density(zI);   % Air density at current altitude
qbar = 0.5 * rho * V^2;

% ---- Update target position ----
target.pos = target.pos + target.vel * dt;

% ---- Missile inertial position ----
pos_missile = [xI; yI; zI];

% ---- GUIDANCE: Proportional Navigation ----
[theta_ref, lambda_dot, lambda_prev] = guidance_law( ...
    pos_missile, X(1:3), theta, psi, phi, ...
    target.pos, target.vel, ...
    guidance.N, lambda_prev, tk, t_prev, dt);

t_prev = tk;

% ---- ADRC CONTROLLER: pitch channel ----
[de, eso_state] = adrc_controller( ...
    theta, q, theta_ref, eso_state, adrc, dt);

% Saturate elevator deflection
de = max(-adrc.de_max, min(adrc.de_max, de));

% ---- DISTURBANCE: wind gust ----
wind = disturbance_model(tk);

% ---- AERODYNAMIC FORCES & MOMENTS ----
[Fx_aero, Fy_aero, Fz_aero] = aerodynamic_forces( ...
    qbar, alpha, beta, missile);

[Mx_aero, My_aero, Mz_aero] = aerodynamic_moments( ...
    qbar, alpha, beta, p, q, r, V, de, missile);

% ---- THRUST ----
if tk <= missile.t_burn
    T = missile.thrust;
else
    T = 0;
end
Fx_thrust = T;

% ---- 6-DOF DYNAMICS: Euler integration ----
Xdot = sixdof_dynamics(X, ...
    Fx_aero + Fx_thrust, Fy_aero, Fz_aero, ...
    Mx_aero, My_aero, Mz_aero, ...
    wind, missile);

X = X + Xdot * dt;

% ---- Normalize Euler angles to [-pi, pi] ----
X(7) = wrapToPi(X(7));
```

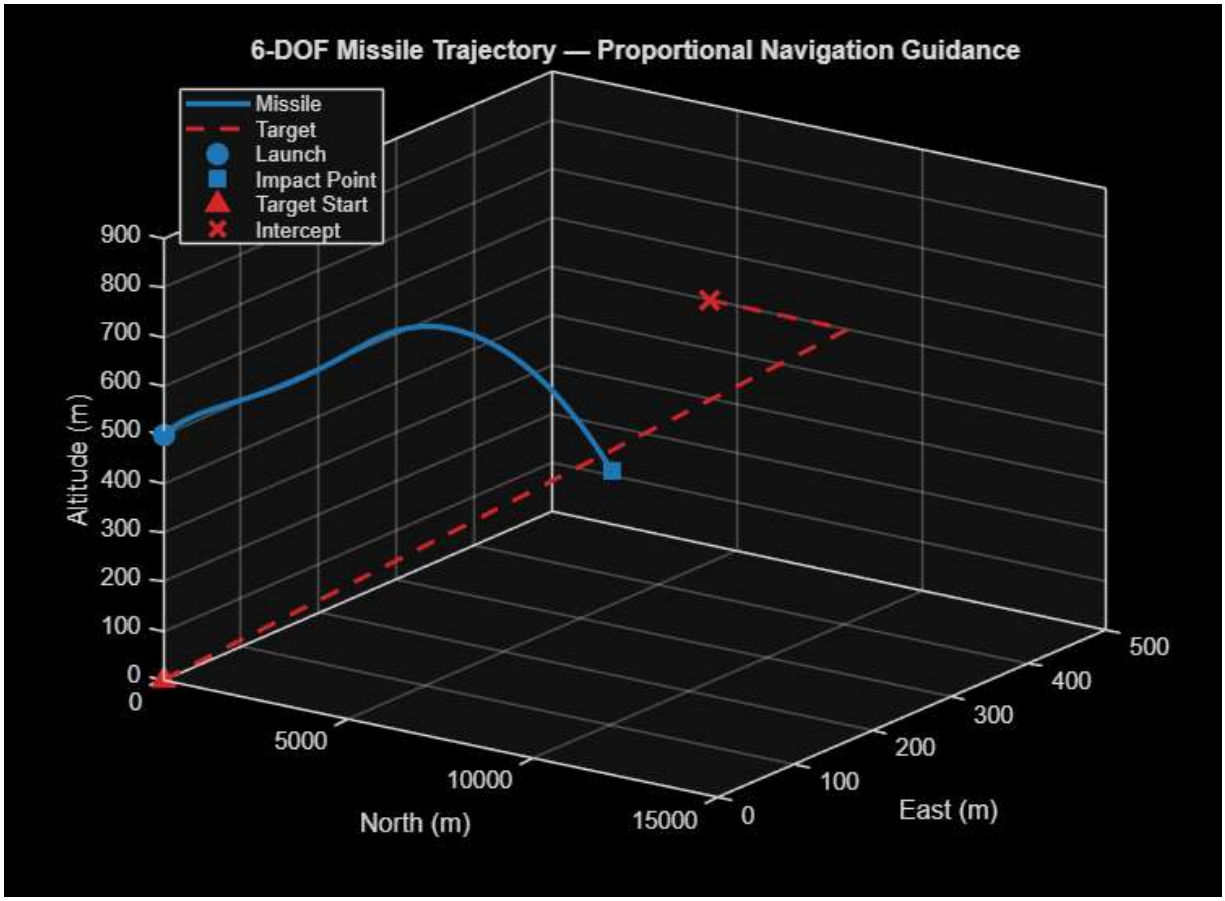
```
    X(8) = wrapToPi(X(8));
    X(9) = wrapToPi(X(9));

    % ---- Store history ----
    X_hist(:, k+1)     = X;
    de_hist(k+1)       = de;
    theta_cmd(k+1)     = theta_ref;
    dist_est(k+1)      = eso_state(3);
    target_hist(:,k+1)= target.pos;
    miss_dist(k+1)     = norm(target.pos - pos_missile);

    % ---- Termination: intercept or ground impact ----
    if miss_dist(k+1) < 20
        fprintf('\n*** TARGET INTERCEPTED at t = %.2f s ***\n', tk);
        fprintf('    Miss distance: %.2f m\n', miss_dist(k+1));
        X_hist      = X_hist(:, 1:k+1);
        de_hist     = de_hist(1:k+1);
        theta_cmd   = theta_cmd(1:k+1);
        dist_est    = dist_est(1:k+1);
        target_hist = target_hist(:, 1:k+1);
        miss_dist   = miss_dist(1:k+1);
        t           = t(1:k+1);
        break;
    end

    if X(12) > 0      % NED: z > 0 means below ground
        fprintf('\n*** GROUND IMPACT at t = %.2f s ***\n', tk);
        X_hist      = X_hist(:, 1:k+1);
        de_hist     = de_hist(1:k+1);
        theta_cmd   = theta_cmd(1:k+1);
        dist_est    = dist_est(1:k+1);
        target_hist = target_hist(:, 1:k+1);
        miss_dist   = miss_dist(1:k+1);
        t           = t(1:k+1);
        break;
    end
end

fprintf('Simulation complete. Duration: %.2f s\n\n', t(end));
```

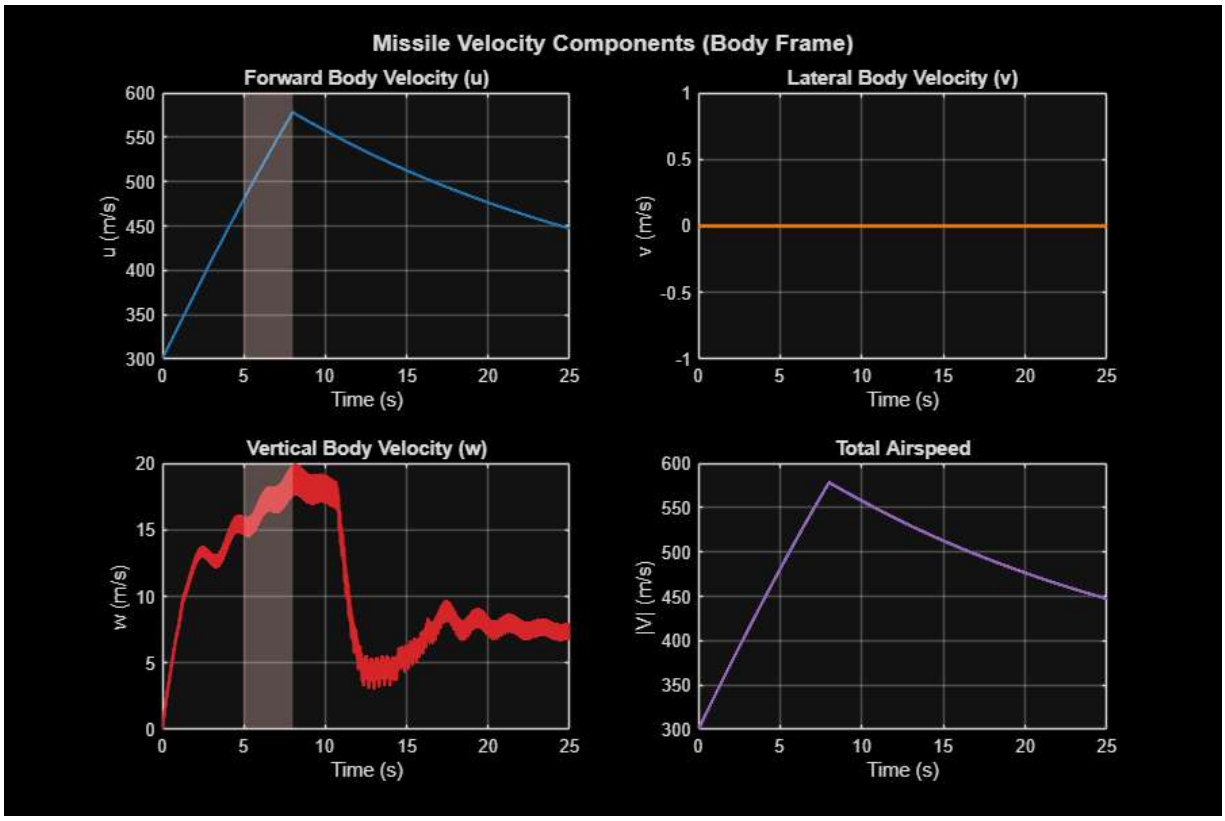
---- PLOT RESULTS

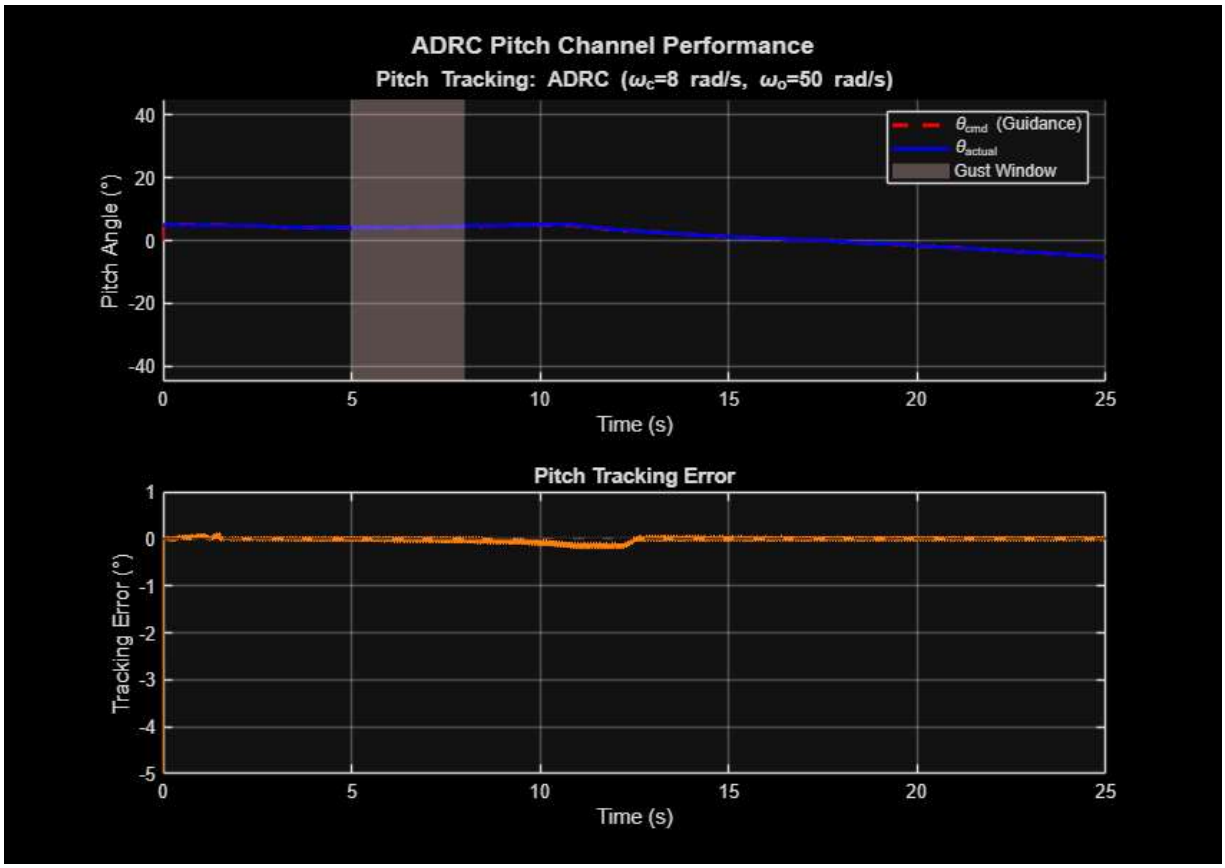
```
plot_results(t, X_hist, de_hist, theta_cmd, dist_est, target_hist,
miss_dist, adrc);
```

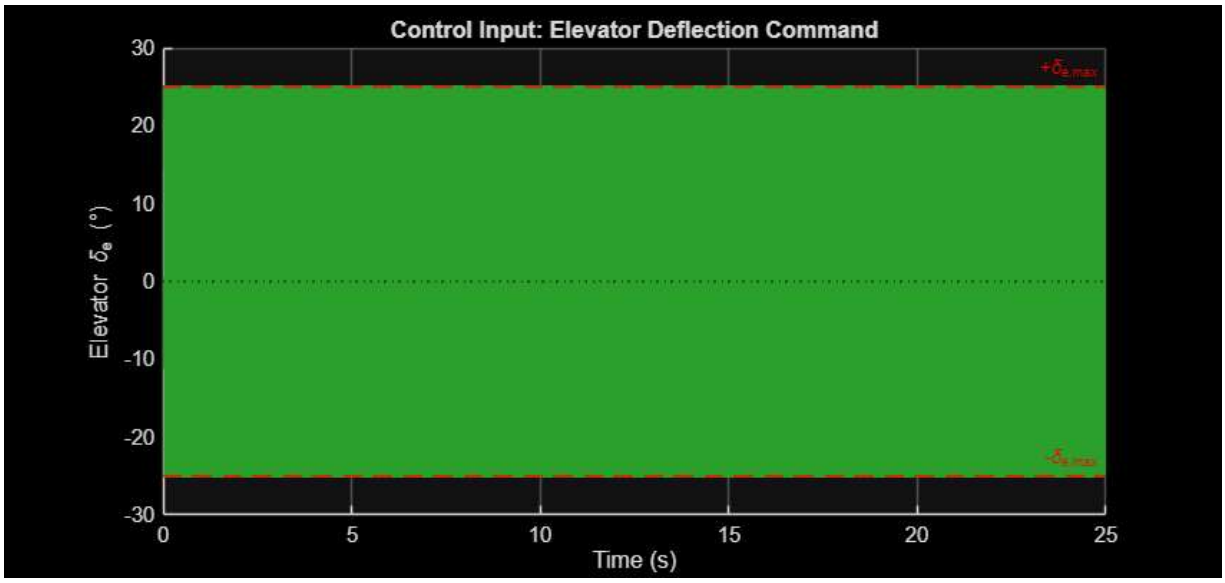
```
Generating plots...

=== SIMULATION SUMMARY ===
Final miss distance:       7881.48 m
Max elevator deflection:   25.00 °
Max pitch angle:           5.32 °
```

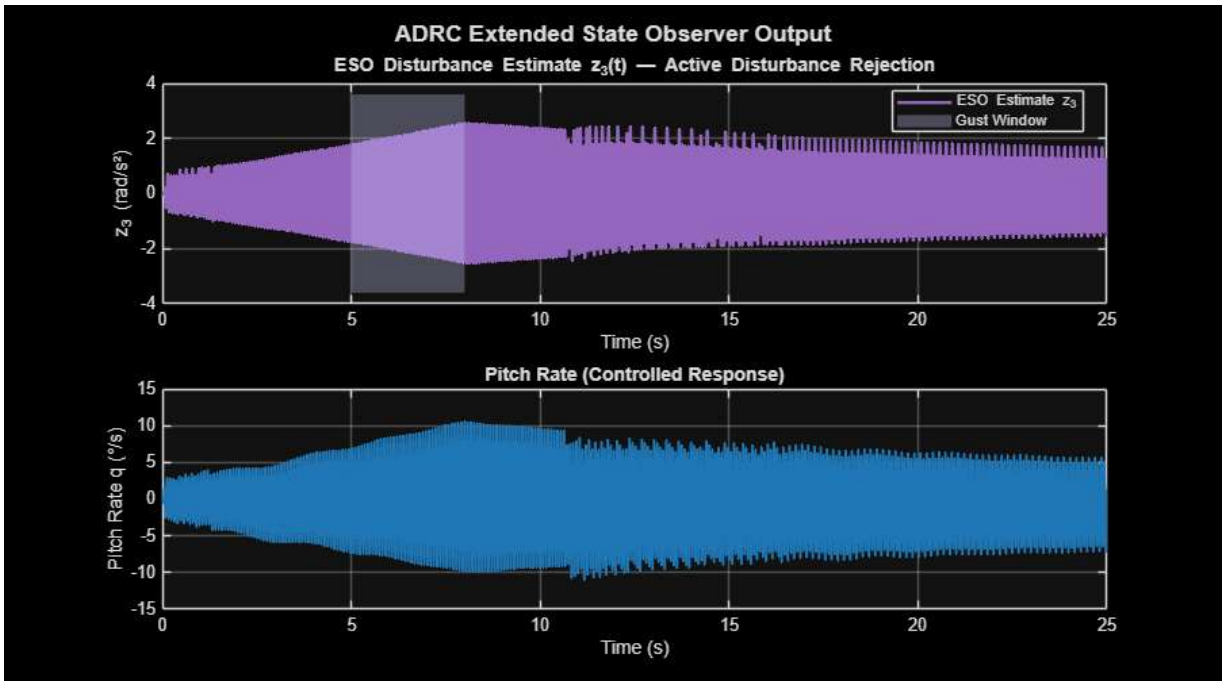
```
Max angle of attack:      2.02 °
Max pitch tracking error: 5.00 °
Max disturbance estimate: 2.56 rad/s²
Peak airspeed:            578.3 m/s (Mach 1.70)
==========================

All figures generated successfully.
```

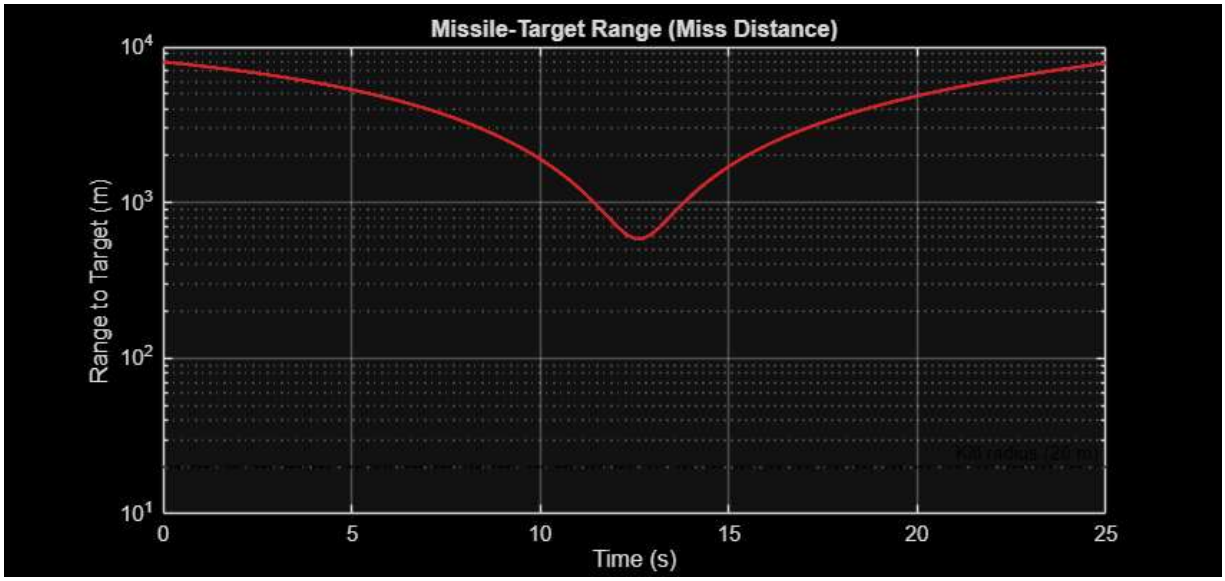


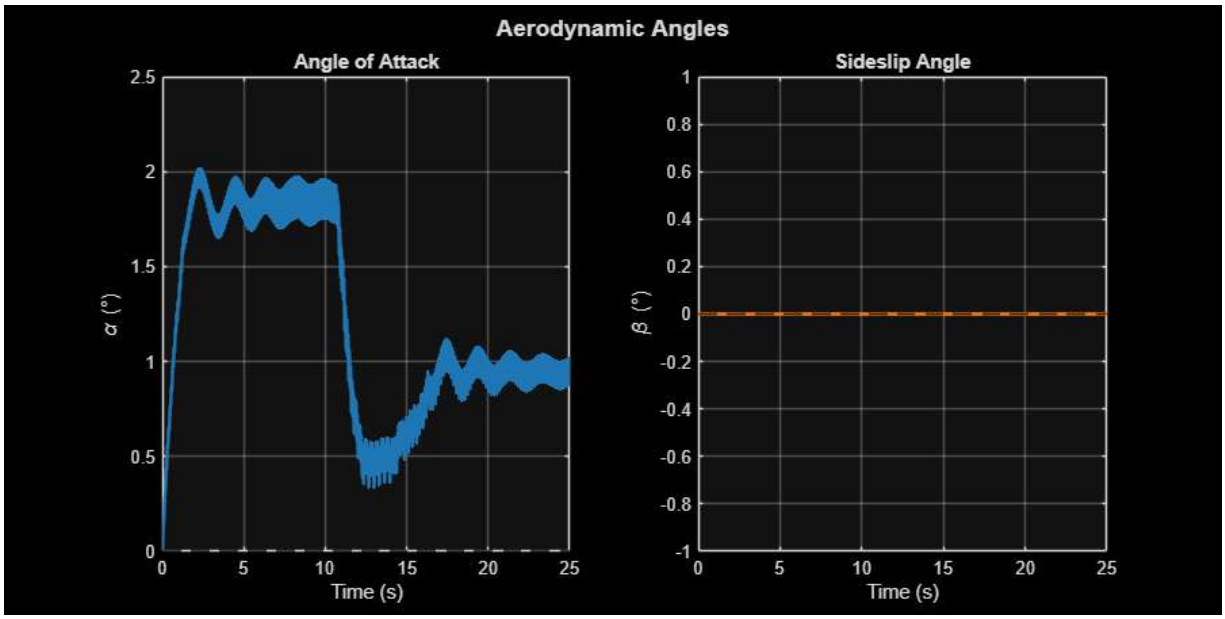

Missile Velocity Components (Body Frame)
Forward Body Velocity (u)
u (m/s)
Time (s)
Lateral Body Velocity (v)
v (m/s)
Time (s)
Vertical Body Velocity (w)
w (m/s)
Time (s)
Total Airspeed
|V| (m/s)
Time (s)

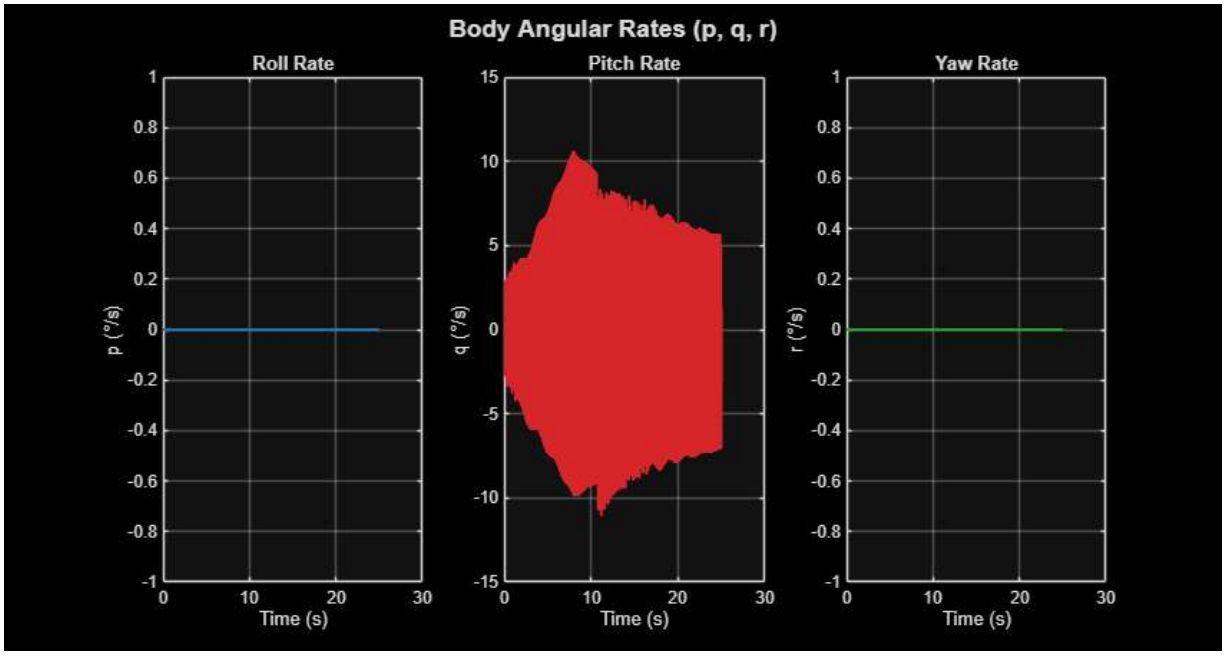

ADRC Pitch Channel Performance
Pitch Tracking: ADRC (ω_c=8 rad/s, ω_o=50 rad/s)
θ_{cmd} (Guidance)
θ_{actual}
Gust Window
Pitch Angle (°)
Time (s)
Pitch Tracking Error
Tracking Error (°)
Time (s)


Control Input: Elevator Deflection Command
$+\delta_{e\,max}$
$-\delta_{e\,max}$
Elevator δ_e (°)
Time (s)


ADRC Extended State Observer Output
ESO Disturbance Estimate z3(t) — Active Disturbance Rejection
ESO Estimate z3
Gust Window
z3 (rad/s²)
Time (s)
Pitch Rate (Controlled Response)
Pitch Rate q (°/s)
Time (s)


Missile-Target Range (Miss Distance)
Range to Target (m)
Time (s)






```
==========================================================

==========================================================

LOCAL HELPER: Standard Atmosphere (simplified ISA model)
Returns air density rho (kg/m³) given altitude z in NED (m, +down)
Altitude above MSL = -z_NED
=====================================================================

function rho = atmosphere_density(z_NED)
    h = max(-z_NED, 0);        % altitude above MSL (m), clamp to ≥ 0
```

```
    T = 288.15 - 0.0065*h;     % temperature (K), valid 0-11 km
    T = max(T, 216.65);         % tropopause
    p = 101325 * (T/288.15)^5.2561;
    rho = p / (287.05 * T);
end
```

Simulation complete. Duration: 25.00 s

Published with MATLAB® R2025a